



The University of Jordan
School of Engineering

Engineering Training Report

Student Name: Farooq Omar Issa Abdullah

Student ID: 0226499

Department: Computer Engineering

Training Institution: PayOne

Training Duration: From 19 July 2026 to 11 September 2026

Date of report submission: 14 September 2026

Table of Contents

Training Summary.....	3
Chapter 1: Institution Profile and Training Outline	4
Chapter 2: Practical Training	6
Chapter 3: Impact of Training.....	12
Chapter 4: Conclusions	14
Chapter 5: Recommendations	15

Training Summary

This report summarizes the eight-week engineering training completed at PayOne from 19 July to 11 September 2026. The placement was in Software Development within an institution whose field is payment systems and payment gateways. The practical project was a Java web application for secure user and role administration. It began as a Servlet/JSP application and was progressively extended into a layered system with database persistence, authentication, authorization, password controls, audit logging, account lockout, self-service password recovery, and a complete JSON REST API.

The main educational objective was to move beyond isolated programming exercises and experience how a maintainable server application is designed, tested, secured, and versioned. I worked with Java 21, Jakarta Servlets, JSP, Maven, Apache Tomcat, Hibernate ORM, PostgreSQL, Jackson, BCrypt, SMTP, JUnit, PowerShell automation, Git, and GitHub. Each phase required me to connect application behavior to HTTP, database, and security concepts rather than merely make a screen appear to work.

The final backend supports registration and login, session-based authentication, role-based permissions, paged and searchable user administration, safe user and role maintenance, password policy and history, password expiry and forced change, account lockout and unlock, administrator password reset, audit log filtering, and forgot-password tokens that are random, time-limited, single-use, and stored only as hashes. The REST API exposes the same capabilities as JSON endpoints without duplicating the underlying business logic. Sensitive fields such as password hashes and internal reset-token data are never placed in API responses.

The most important result was architectural rather than visual. Business rules were moved into service classes used by both the JSP servlets and the API servlet. This means a future Angular client can call the API while receiving the same validation, authorization, and security behavior as the existing interface. At the end of the period the Maven build produced a deployable WAR and the automated test suite passed twelve tests. Live Gmail SMTP configuration, Angular implementation, and production hosting remain deployment activities rather than completed claims in this report.

Chapter 1: Institution Profile and Training Outline

1.1 Training Institution and Field

PayOne operates in the field of payment systems and payment gateways. Software used in this field must treat identity, access control, traceability, and failure handling as core requirements because the surrounding business context depends on controlled access and reliable records. My placement was in Software Development, where the assigned work focused on a secure administration application rather than on direct processing of financial transactions.

The training was supervised by Abdel Raouf Ghanem at the PayOne site on Al Amira Basma Street in Amman. The working arrangement resembled a small software-delivery cycle: requirements were provided in phases, an implementation was produced, behavior was tested, feedback was reviewed, and the next phase reused the previous one. This made early design choices visible later. For example, moving authentication, password, user, role, and audit operations into services made it practical to add a REST API without reproducing every rule.

1.2 Training Objectives

The training objectives were to understand the complete path of a web request, apply a layered design, connect a Java application to a relational database, implement defensive authentication and authorization, and use professional source-control practices. A second objective was to learn how an existing application evolves. The project was not rewritten when requirements changed; it was refactored so new interfaces could reuse verified functionality.

- Develop the application with Java Servlets and JSP and deploy it through Apache Tomcat.
- Persist users, roles, permissions, password history, reset tokens, and audit records in PostgreSQL through Hibernate.
- Apply validation and security rules consistently at the HTTP, service, and persistence boundaries.
- Expose the completed functionality as a REST API suitable for a later Angular frontend.
- Build, test, document, and version the work with Maven, JUnit, Git, and GitHub.

1.3 Eight-Week Training Outline

Week	Primary focus	Main outcome
1	Project setup and environment	Maven web project, Tomcat deployment, PostgreSQL connection, Git workflow
2	Users and persistence	Hibernate entities, DAOs, registration and basic user administration
3	Authentication and authorization	BCrypt login, sessions, filters, roles and permissions
4	Password lifecycle	Policy, history, expiry, forced change, administrative reset
5	Administration and audit	Paging/search/sorting, audit records and safer management rules
6	Recovery and protection	Lockout, unlock, hashed reset tokens and SMTP-based recovery design
7	REST API foundation	JSON routing, authentication filter, safe view mapping and unified errors
8	REST API completion	All required endpoints, shared services, documentation, tests and WAR build

Chapter 2: Practical Training

2.1 Development Environment and Request Flow

The project used Maven to manage dependencies and produce a WAR file for Tomcat. Jakarta Servlet classes received HTTP requests, while JSP files rendered the original browser interface. Hibernate mapped Java entities to PostgreSQL tables. Environment variables were used for values that should not be committed, including database and mail credentials and the public reset URL. PowerShell scripts assisted with local environment loading, building, deployment, database checks, and stopping Tomcat.

A normal request passes through several responsibilities. Tomcat creates `HttpServletRequest` and `HttpServletResponse` objects and invokes the matching servlet method. A filter can reject an unauthenticated or unauthorized request before it reaches the servlet. The servlet reads transport data, calls a service, and selects a view or writes JSON. The service enforces business rules and coordinates DAOs. Hibernate then generates database operations. Keeping these steps separate made failures easier to locate and prevented web pages from containing database or password logic.

Layer	Responsibility	Examples from the project
Presentation / transport	Accept HTTP input and produce JSP or JSON output	JSP servlets, <code>ApiServlet</code> , <code>ApiResponses</code>
Security filters	Apply session and forced-password-change gates	<code>ProtectedEndpointFilter</code> , <code>ApiAuthenticationFilter</code>
Service	Enforce reusable use cases and business rules	<code>AuthenticationService</code> , <code>UserManagementService</code> , <code>RoleManagementService</code>
DAO / ORM	Query and update persistent data	<code>HibernateUserDao</code> , <code>HibernateRoleDao</code> , <code>HibernateAuditLogDao</code>
Database	Store application state and constraints	PostgreSQL users, roles, permissions, history, tokens, audit log

2.2 Registration, Login, and Session Security

Registration validates the username, email, and password before creating a user. Passwords are not stored as plain text; BCrypt creates a salted one-way hash. During login, `AuthenticationService` retrieves the account and verifies the submitted password against that hash. A successful result establishes the HTTP session and stores the authenticated user identity. A failed attempt is recorded, and repeated failures can temporarily lock the account according to `LoginPolicy`.

Session-based authentication was retained for the API. After POST /api/auth/login succeeds, the browser or Angular client keeps Tomcat's session cookie and sends it on later requests. ApiAuthenticationFilter covers /api/*, allows only the public authentication and recovery routes without a session, and returns a JSON 401 response when authentication is absent. If a password change is required, the filter permits only logout and the password-change endpoint until the user completes that action.

```
@WebFilter("/api/*")
public class ApiAuthenticationFilter implements Filter {
    // Public routes pass; protected routes require a valid session.
    // Forced password change limits the routes available to the user.
}
```

2.3 Role-Based Access Control

The authorization model separates roles from permissions. A user receives a role, and each role contains allowed functions such as viewing users, editing users, deleting users, managing roles, resetting passwords, unlocking accounts, or viewing the audit log. PermissionAccess checks the authenticated actor before a protected operation is performed. This is more maintainable than scattering checks for role names throughout the program because behavior is tied to explicit capabilities.

Administrative safeguards also protect system availability. FullAdminProtection prevents operations that would remove the final fully privileged administrator. UserManagementService and RoleManagementService apply this rule for both JSP and API callers. Therefore changing the transport layer cannot bypass it. Unauthorized requests receive 403 Forbidden, missing records receive 404 Not Found, and protected or conflicting operations receive 409 Conflict where appropriate.

2.4 User Directory, Paging, Search, and Sorting

The user list was extended to support paging, text search, and controlled sorting. A UserPageRequest carries page, search, sort, and direction values into UserDirectoryService. The service validates acceptable sort fields and direction values before the DAO builds the query. User-supplied values are bound as parameters rather than concatenated into query text. This improves correctness, limits injection risk, and gives the JSP screen and REST endpoint identical results.

The REST form is GET /api/users?search=...&sort=...&dir=...&page=.... Its response contains safe user items plus paging metadata. The API mapper intentionally creates response objects instead of serializing Hibernate entities. This prevents accidental exposure of password hashes and prevents ORM relationships or lazy-loaded internals from leaking into JSON.

2.5 Password Lifecycle Management

PasswordManagementService coordinates all password changes. PasswordPolicy validates minimum strength, and password history prevents immediate reuse of recent credentials. The current password is verified for a self-service change. Administrative reset follows a separate permission-controlled route and can force a password change at the next login. Password timestamps support expiry behavior, while audit records describe the actor, target, action, success state, and contextual details.

These rules were placed in the service layer because they must remain true whether the request originated from a JSP form or a JSON client. The servlet's job is to convert HTTP input into a service call; it must not become an alternative implementation of password security.

2.6 Account Lockout and Audit Logging

LoginPolicy tracks repeated authentication failures and applies a temporary lock. The public response remains deliberately general so it does not reveal unnecessary account information. A permitted administrator can unlock an account through the JSP interface or POST /api/users/{id}/unlock. Successful and unsuccessful security-relevant actions are written through the audit service.

The audit log supports pagination and filtering by action, actor, target type, success state, and page. AuditLogService validates the filter and delegates to HibernateAuditLogDao. GET /api/audit-log exposes the same records to authorized clients using safe response mapping. The result gives administrators a trace of actions without granting ordinary users access to security history.

2.7 Secure Forgot-Password Flow

The forgot-password process uses a random reset token as a temporary bearer credential. The raw token is placed in the email link because the user must present it later. Before persistence, ResetTokenHasher derives a one-way hash; only that hash, the associated user, expiry, and used state are stored. When the link is submitted, the server hashes the presented raw token again and searches for the matching stored value. A valid token must also be unexpired and unused. After a successful reset it is marked used, so replaying the same link fails.

SMTP does not decrypt or validate the token. MailService merely transports the link. The safety comes from token entropy, HTTPS during real deployment, limited lifetime, single-use enforcement, and storing no reusable raw token in the database. An attacker who changes a user identifier or token in

the URL cannot create a matching hash. The application obtains the frontend reset address from an environment variable so local JSP testing and a future Angular deployment can use different base URLs without changing code.

```
rawToken = generateSecureRandomToken();
storedHash = ResetTokenHasher.hash(rawToken);
save(userId, storedHash, expiry, used = false);
sendMail(frontendResetUrl + "?token=" + rawToken);
// Completion: hash the submitted raw token and validate hash, expiry, and used state.
```

2.8 REST API Design and Implementation

The final phase exposed the existing operations under /api as JSON endpoints. ApiServlet is mapped with `@WebServlet("/api/*")` and implements `doGet`, `doPost`, `doPut`, and `doDelete`. The request method expresses the intended operation: GET reads data, POST creates resources or performs commands, PUT updates a complete resource representation, and DELETE removes a resource. Route parsing identifies whether the request concerns authentication, users, roles, functions, or audit data.

Jackson deserializes request JSON into small request records and serializes maps or response records into JSON. ApiResponse centralizes content type, character encoding, parsing, writing, and the error structure. Tomcat does not wait for a returned Java value because servlet methods are void; the servlet writes to the supplied HttpServletResponse. When doPost or another handler returns, Tomcat finalizes and sends the status, headers, and response body that were written into that object.

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException, ServletException {
    LoginRequest body = ApiResponse.read(request, LoginRequest.class);
    AuthenticationResult result = authenticationService.authenticate(...);
    ApiResponse.write(response, HttpServletResponse.SC_OK,
        Map.of("user", ApiViewMapper.user(result.user())));
}
```

The response mapper is a security boundary. ApiViewMapper copies only fields intended for clients, such as identifier, username, email, role summary, lock state, and relevant dates. It never returns password hashes, reset-token hashes, or raw persistence entities. A unit test verifies that sensitive user fields are absent from serialized output.

2.9 Endpoint and Status-Code Summary

Operation	Endpoint	Method / success
Register	/api/auth/register	POST / 201
Login, logout, current user	/api/auth/login, /logout, /me	POST, POST, GET / 200
Forgot / complete reset	/api/auth/forgot-password, /reset-password	POST / 200
List / read users	/api/users, /api/users/{id}	GET / 200
Create / update / delete user	/api/users, /api/users/{id}	POST / 201; PUT, DELETE / 200
Own password	/api/users/me/password	POST / 200
Admin reset / unlock	/api/users/{id}/reset-password, /unlock	POST / 200
Role management	/api/roles, /api/roles/{id}	GET, POST, PUT, DELETE / 200 or 201
Functions	/api/functions	GET / 200
Audit log	/api/audit-log	GET / 200

The API uses 400 Bad Request for invalid input or JSON, 401 Unauthorized when authentication is missing or invalid, 403 Forbidden when the actor lacks permission, 404 Not Found for unknown endpoints or resources, and 409 Conflict for protected or conflicting changes. Error responses use one JSON shape containing an error code and human-readable message. This consistency lets an Angular client handle failures in one place instead of guessing the meaning of different formats.

2.10 Shared Services and Avoidance of Duplicated Logic

The most significant refactoring was making the JSP servlets and ApiServlet depend on the same services. `UserManagementService` owns registration, creation, editing, deletion, and administrative user rules. `RoleManagementService` owns role validation, permission assignment, update, deletion, and administrator protection. `AuthenticationService`, `PasswordManagementService`, `ForgotPasswordService`, `UserDirectoryService`, and `AuditLogService` perform their corresponding use cases.

This structure addresses a common risk during API development. If the API had copied rules from the JSP servlets, later corrections might be applied to one interface but not the other. With shared services, the interface decides only how to read the request and represent the response. Validation, authorization, transactions, and domain decisions remain authoritative in one place.

2.11 Verification, Documentation, and Version Control

Maven was used to compile the application, execute tests, and package the WAR. The completed build passed twelve automated tests covering password policy, login policy, final-administrator protection, reset-token hashing, user and role validation, and safe API view mapping. Manual verification was also performed through the local Tomcat application and PostgreSQL environment. The REST contract was documented in docs/rest-api.md with authentication behavior, endpoint tables, status codes, query parameters, and sample requests.

Git and GitHub were used throughout the work. Feature and practice branches demonstrated staging, commits, upstream tracking, merge conflicts, stashing, restoration, and log inspection. Meaningful application changes were committed to main and pushed to the remote repository. This workflow provided a recoverable history and reinforced the difference between the working tree, staging area, local commits, and the remote repository.

Chapter 3: Impact of Training

3.1 Technical Knowledge

The training changed my understanding of web development from a collection of pages into a sequence of controlled boundaries. I can now trace how a browser or Angular client creates an HTTP request, how Tomcat chooses a filter and servlet, how JSON or form data is parsed, how services enforce rules, how Hibernate interacts with PostgreSQL, and how the response is returned. Concepts such as status codes, cookies, sessions, password hashing, token hashing, ORM mapping, and dependency management became concrete because each affected working code.

I also learned that security is not produced by one library call. BCrypt protects stored passwords, but a secure account system also needs input validation, authorization, password history, lockout, audit trails, token expiry, single-use recovery links, safe error behavior, environmental secret management, and response mapping. The project showed how these controls reinforce each other.

3.2 Software Engineering Skills

The strongest software-engineering lesson was the value of separation of concerns. Earlier behavior could work correctly while still being difficult to extend. Extracting service classes reduced coupling between business rules and JSP screens, and that investment made the REST phase possible. I learned to judge code not only by whether it runs today but also by whether another interface can reuse it safely tomorrow.

Debugging skills improved through reading stack traces, checking HTTP responses, inspecting Git state, isolating database configuration, and testing small policy classes. Version-control exercises made branching and conflicts less abstract. A branch is a movable reference to a commit history, while a merge combines histories and may require a person to decide between incompatible edits. Stashing provides temporary storage for unfinished tracked changes; it is not a replacement for a meaningful commit.

3.3 Relation to Computer Engineering

The placement supported my Computer Engineering studies by linking programming, databases, computer networks, operating systems, and information security. HTTP request handling relates application behavior to network protocols; Tomcat and JVM execution connect software to processes and resource management; PostgreSQL transactions and constraints apply database theory; hashing and

authorization apply security principles; and Git models distributed state and collaboration. Seeing these subjects interact in one deployable system was more valuable than studying each in isolation.

3.4 Professional Development

The phased requirements improved my ability to estimate work, ask what a requirement protects, and explain design choices. I became more comfortable reading unfamiliar code, refactoring without discarding working behavior, documenting an API for another frontend developer, and distinguishing completed implementation from deployment work that still needs infrastructure or credentials. These habits are directly relevant to collaborative software teams.

Chapter 4: Conclusions

The PayOne training provided a useful progression from environment setup to a complete backend interface. The final system demonstrates secure user administration through both JSP pages and REST JSON endpoints. It includes database persistence, authentication, role-based permissions, password controls, recovery tokens, lockout, auditability, consistent HTTP behavior, safe response mapping, documentation, and automated verification.

The project also demonstrated why architecture matters. Everything might appear satisfactory while a single JSP interface is the only caller, but placing rules inside page controllers makes a second client expensive and dangerous. The service-layer refactor created one authoritative implementation that can serve multiple transports. ApiServlet and the JSP servlets now differ in how they receive input and return output, while the security and management decisions remain shared.

The training objectives were substantially achieved. I can build and explain the request lifecycle, database access, authentication flow, authorization checks, reset-token verification, REST routing, JSON serialization, HTTP status choices, testing, and Git workflow. The main remaining tasks are operational: configure and verify Gmail SMTP with real credentials, deploy the backend and PostgreSQL database to hosted infrastructure, and build the planned Angular client against the documented API.

Overall, the placement was technically relevant and appropriately cumulative. Feedback in each phase directly influenced the following phase, particularly the emphasis on auditability, password security, and reusable services. The resulting project is a strong foundation for frontend replacement and production-oriented improvements.

Chapter 5: Recommendations

5.1 Recommendations for the Application

- Deploy the backend and PostgreSQL database to controlled infrastructure and require HTTPS so session cookies and reset tokens are protected in transit.
- Store database, SMTP, and deployment secrets in a managed secret store rather than source files, and use separate development and production configuration.
- Complete live Gmail SMTP testing with an application password or replace it with a transactional mail provider designed for production delivery.
- Implement the Angular client against the documented JSON API, configure allowed origins carefully, and keep credentialed cross-origin requests restricted to known frontend addresses.
- Add broader integration tests for servlet routes, permissions, Hibernate transactions, concurrent administrative changes, lockout timing, and complete reset-token consumption.
- Introduce CI/CD checks that run the Maven test and package phases before merging or deploying, and retain deployable build artifacts with version identifiers.
- Add structured server logging, monitoring, database backup procedures, and alerting for repeated authentication failures or abnormal administrative activity.
- Consider API versioning, request-rate limits for login and recovery routes, and documented production cookie attributes before public release.

5.2 Recommendations for Future Training

Future trainees would benefit from receiving the staged requirements together with a short acceptance checklist for each phase. The existing progression should be retained because later requirements reveal the value of earlier refactoring. A final joint review covering deployment, code review, and threat modeling would complement the strong implementation focus.

It would also be useful to pair the backend work with a small frontend integration task and a controlled deployment environment. That would allow trainees to observe CORS, cookies, HTTPS, environment configuration, database migration, email delivery, and monitoring as one system. These are the areas where local development differs most from a globally accessible application.

For future students, I recommend maintaining brief daily notes, committing focused changes with clear messages, checking git status and staged diffs before each commit, and documenting decisions while the reason is still fresh. These habits reduce report-writing effort and make technical handover more accurate.